



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Ángel Julián García López **Carné:** 1097925

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
R/ Es una interfaz que permite que diferentes aplicaciones se comuniquen mediante HTTP.
2. ¿Qué es un endpoint?
R/ Es una URL específica donde una API recibe o envía información.
3. ¿Qué diferencia existe entre una URL y un endpoint?
R/ Una URL identifica un recurso en Internet; un endpoint es una URL específica de una API para realizar una función.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
R/ GET: obtener datos.
POST: crear datos.
PUT/PATCH: actualizar datos.
DELETE: eliminar datos.
5. ¿Qué es JSON y por qué es común en las APIs?
R/ Es un formato para organizar datos. Es común porque es sencillo, ligero y fácil de leer.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
R/ Indica el resultado de una solicitud. Ejemplos: 200 = correcto, 404 = no encontrado, 500 = error del servidor.
7. ¿Qué información se puede encontrar en la respuesta de una API?
R/ Datos solicitados, código de estado, mensajes y, en ocasiones, información sobre errores.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
R/ Permite probar solicitudes, revisar respuestas y detectar errores antes de programar.
9. ¿Qué es y para qué sirve Swagger UI?
R/ Es una herramienta que permite visualizar y probar la documentación de una API.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
R/ Sirve para desarrollar, probar y verificar el funcionamiento de APIs.



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

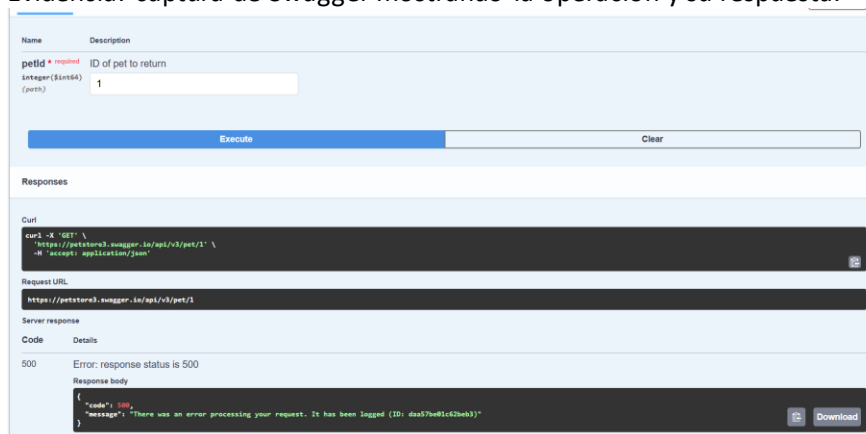
Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingrese a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
`https://petstore3.swagger.io/api/v3/{petId}`
- Indique el método HTTP utilizado.
GET
- Indique qué parámetro necesita la solicitud.
petId
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
`https://petstore3.swagger.io/api/v3/pet/1`
- Explique con sus palabras qué información contiene la respuesta.
Debería de contener la información del pet con id numero "1"

Evidencia: captura de Swagger mostrando la operación y su respuesta.



3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id

name



Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
 - GET – Consultar mascota por ID
 - POST – Crear mascota
 - PUT – Modificar mascota
 - GET – Buscar por status
 - DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.



- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

The screenshot shows a REST client interface with a GET request to `/pet/{petId}` (where `petId` is 10). The response is a 200 status code with a successful operation. The media type is set to `application/json`. The example value is a JSON object:

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

On the right, the Chrome DevTools Network tab is visible, showing a list of requests including `petstore3.swagger.io` and `swagger-ui.css`.

Entrega

Dentro del repositorio de GitHub, cree la carpeta `semana-13/lab1/` y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.